

SIM_LGP30 – Supplementary Details

Author: Neil Higgins

Applicable to: SIM_LGP30 Version 1.0

Contents

Contents.....	1
Disclaimer.....	2
LGP-30 Operation.....	2
Multiplication, Division, Input and Bits.....	4
More Details of Multiplication and Division	6
Multiplication.....	6
Division.....	7
More on LGP-30 Arithmetic	7
On Reading Tape Files.....	7
Simulator Implementation.....	8
LGP-30 Programming.....	8
References	9

Disclaimer

This document and the SIM_LGP30 simulator which it describes are free (as in beer) and free (as in thought). Do with them as you will, provided that you accept all of the consequences.

LGP-30 Operation

Before an opcode is dispatched, an instruction fetch takes place, i.e. the instruction at C/pc is read into R/ir, and C/pc is incremented.

The fetched instruction is decoded to determine which (if any) operands should be read, what operation should be performed, and which (if any) results should be written. This is summarised below:

Order	Operation
Fetch	$R/ir < \text{mem}(C/pc)$ $C/pc += 1$
b	$A/ac < \text{mem}(\text{OpAd})$
h	$\text{mem}(\text{OpAd}) < A/ac$ (31 bits)
c	$\text{mem}(\text{OpAd}) < A/ac$ (31 bits) $A/ac < 0$
a	$A/ac < A/ac + \text{mem}(\text{OpAd})$
s	$A/ac < A/ac - \text{mem}(\text{OpAd})$
m	$A/ac < A/ac \text{ m} * \text{mem}(\text{OpAd})$
n	$A/ac < A/ac \text{ n} * \text{mem}(\text{OpAd})$
d	$A/ac < A/ac / \text{mem}(\text{OpAd})$
e	$A/ac < A/ac \& \text{mem}(\text{OpAd})$
u	$C/pc < \text{OpAd}$
t	$T < R/ir$ (sign bit) & TCS $C/pc < \text{OpAd}$ if (A/ac (sign bit) or T)
y	$\text{mem}(\text{OpAd})$ (address portion) < A/ac (address portion)
r	$T < C/pc + 1$ $\text{mem}(\text{OpAd})$ (address portion) < T
p	$O < A/ac$ (track portion)
i	$A/ac < \text{lsh}(A/ac, 4 \text{ or } 6)$ $A/ac < A/ac \mid I$
Z	$T < R/ir$ (track portion) & Switches Halt if $T = 0$

A/ac: Accumulator
C/pc: Counter
I: Input
m*: m multiply
mem(<address>): Value stored in memory at <address>
n*: n multiply
O: Output
OpAd: Operand address portion of R/ir
R/ir: Instruction Register
Switches: Break Point Switches
T: Intermediate value
TCS: Transfer Control switch

In all cases involving memory operands, latency may be incurred until the required sector appears under the read/write head. In the LGP-30, 64 read signals must be multiplexed to a single serial input, and a single serial output must be demultiplexed to one of 64 write heads.

There is no latency for C/pc, R/ir and A/ac.

The LGP-30 is a serial machine, meaning that for simple operations the result can be written while the operand(s) are being read. This is not the case for instructions such as multiply and divide, which take more than one machine cycle. For the y and r instructions, a read-modify-write operation is avoided by overwriting the address portion of the operand in-place.

SIM_LGP-30 does not simulate serial operation. Instructions are implemented in a read-then-modify-then-write fashion, rather than in a read-and-modify-and-write fashion.

Details of the instruction fetch / execute cycle are available in LGP-30 documents such as the Operations Manual, the Programming Manual [1] and the Programming Class Notes [2].

Multiplication, Division, Input and Bits

The LGP-30 documentation [1, 2, 3, 4] covers these topics using somewhat arcane terminology and typography. The following is a somewhat updated presentation of the essentials.

The LGP-30 treats numbers as fixed-point fractions. The decimal point nominally resides between the sign bit and the first magnitude bit, i.e. the first magnitude bit has a value of 2^{-1} or 0.5. Numbers comprise a mantissa only.

The exponent is what the programmer chooses it to be, and the machine has no part in this decision.

The decision (regarding the exponent) is embodied in a number called “q”, which is the number of places to the right of the sign bit at which the programmer decides the decimal point should reside. Hence:

S100 0000 0000 0000 0000 0000 000X where S is the sign bit (assumed to be 0/positive for simplicity) and X is the spacer bit, represents (with the points shown for clarity):

S.100 0000 0000 0000 0000 0000 000X, 0.5 at q=0, or

S1.00 0000 0000 0000 0000 0000 000X, 1 at q = 1, or

S10.0 0000 0000 0000 0000 0000 000X, 2 at q = 2, or

S100. 0000 0000 0000 0000 0000 000X, 4 at q = 3, and so on.

Each place on either side of the point represents a power of 2, positive to the left and negative to the right. Hence:

S000 000.0 0100 0000 0000 0000 000X

Represents 2^{-3} or 0.125 q = 6

The rules for addition, multiplication and division are the same rules we learned at high school for logarithms:

- When adding numbers, their q values (exponents) must be the same
- When multiplying a at q_a by b at q_b , the product is a.b at $q_a + q_b$
- When dividing a at q_a by b at q_b , the quotient is a/b at $q_a - q_b$

A simple example:

S011. 0000 0000 0000 0000 0000 000X, 3 at q = 3, times

S001 0.000 0000 0000 0000 0000 000X, 2 at q = 4, results in

S000 0110. 0000 0000 0000 0000 000X, 6 at q = 7

The next step is to consider what happens in a more extreme example:

S000 0000 0000 0011. 0100 0000 0000 000X, 3.25 at q = 15, times

S000 0000 0000 0010. 0000 0000 0000 000X, 2 at q = 15. The result should in principle be

S000 0000 0000 0000 0000 0000 110.1, 6.5 at q = 30, but there is a problem.

The problem is that the fractional part lands in the spacer bit. Even though A/ac is a 32 bit register, the result cannot be saved to the drum without a loss of precision.

Another problem is that on reading a hexadecimal paper tape, it is possible to fill the spacer bit of A/ac with a digit, effectively increasing its precision. In this case the multiplicand will be a 32-bit (sign plus 31 bits) twos complement number. The multiplier will effectively be a 31-bit (sign plus 30 bits) twos complement number since the spacer bit is always zero. In this case the product will be a 62 bit (sign plus 61 bits) twos complement number.

The LGP-30 solves these problems by having two multiply instructions:

M multiply multiplies A/ac by the other operand, leaves the sign plus the most significant 30 bits of the result in A/ac, and sets the spacer bit to zero.

N multiply multiplies A/ac by the other operand, and leaves the least significant 31 bits of the result in A/ac.

Hence for multiplication of 3.25 at $q = 15$ by 2 at $q = 15$:

M multiply produces:

S000 0000 0000 0000 0000 0000 110.X, 6 at $q = 30$

N multiply produces:

.1000 0000 0000 0000 0000 0000 000X, 0.5 at $q = 30$

Note that the most significant bit of the N multiply result is not a sign bit, but one bit of precision.

The concatenation of these two values (ignoring the zero spacer bit in A) is 6.5 at $q = 30$, the correct result with no loss of precision.

We're not done yet. In theory, multiplication of the 32-bit A/ac (sign plus 31 bits) by a 31-bit memory operand (sign plus 30 bits) can produce a 63-bit product (sign plus 62 bits). How does the LGP-30 get by with just 62 bits (sign plus 30 bits for the M multiply and 31 bits for the N multiply)? The answer lies in the fixed-point fractional representation used. This represents the operands as follows:

A/ac: $\pm 0.a_{31}$ (sign plus 31 fractional bits), and

memory operand: $\pm 0.m_{30}$ (sign plus 30 fractional bits),

the exact product of which has $31 + 30 = 61$ fractional bits. Because both operands are fractions, the product cannot overflow into the whole numbers. And the sign bit of the product makes 62.

In the case of division, the quotient cannot run into a second word, so only a single divide instruction is required. However the quotient will overflow if, by magnitude, the dividend is greater than the divisor – this will cause the machine to halt.

One last point in this regard: The existence of floating point libraries for the LGP-30 is a clear indication that, even in those days, users were not fussed on keeping track of exponents manually. They were more than happy to trade off precision and speed in order to relieve themselves of this task. To this day, 32-bit ("single precision") floating point numbers are still regarded as acceptable for many purposes.

For input, the 32-bit width of A/ac is more evident. That's because when a code is received, A/ac is shifted 4 or 6 bits to the left, and the new code is loaded into the 4 or 6 least significant bits. It is bizarre but true that the most recently received code cannot be retrieved in full without shifting A/ac to the left.

Finally, given that the LGP-30 doesn't have shift instructions, how is bit shifting achieved? The answer is simple in principle: Left shift by multiplying by 2, and right shift by dividing by 2. For the LGP-30 there is a twist: S.100 0000 0000 0000 0000 0000 0000 represents 0.5 at $q = 0$, so an M multiply by 0.5 at $q = 0$ (or 1 at $q=1$ if you prefer) *divides* by 2 – a right shift. An N multiply by 1 at $q = 30$ has the effect of a left shift (which you can verify by long multiplication if you wish).

Generalising (this is important when lining up q values for addition and subtraction):

Left Shift		Right Shift	
N multiply		M multiply	
Bits	Factor	Bits	Factor
1	1 at $q = 30$	1	1 at $q = 1$
2	1 at $q = 29$	2	1 at $q = 2$
3	1 at $q = 28$	3	1 at $q = 3$
4	1 at $q = 27$	4	1 at $q = 4$
etc.	...	etc.	...

More Details of Multiplication and Division

Multiplication

Python represents numbers variously as infinite precision integers, fixed-precision integers and floats, converting integers to floats for mixed arithmetic and standard division. When performing bitwise logical operations, Python treats integers as infinite precision, twos complement numbers. All of this was enough to put this author's brain into a spin when considering how to model LGP-30 numbers for the purposes of multiplication and division.

A/ac: $\pm 0.a_{31} = S.nnn\ nnnn\ nnnn\ nnnn\ nnnn\ nnnn\ nnnn\ nnnn$

memory operand: $\pm 0.m_{30} = S.nnn\ nnnn\ nnnn\ nnnn\ nnnn\ nnnn\ nnnn\ nnnX$

SIM_LGP30 multiplication treats both operands as integers.

Python A/ac = Snnn nnnn nnnn nnnn nnnn nnnn nnnn nnnn (as represented internally).

Python memory operand = Snnn nnnn nnnn nnnn nnnn nnnn nnnn nnn0 (as represented internally).

Superfluous leading zeros or ones are suppressed.

Conceptually, the point in the Python representation is left shifted 31 places from its position in the LGP-30 representation, and the memory operand acquires an extra bit of precision (albeit a zero).

Python computes $(\pm 0.a_{31} * 2^{31}) * (\pm 0.m_{30} * 2^{31})$ in 64 bits.

One LSB can be removed to compensate for the anomalous extra precision of the memory operand. Another can be removed for fractional multiplication. That leaves 62 bits, which can be split 31/31 across the results of M multiply and N multiply, with a spacer bit added to each.

From the 64-bit product:

M mask = 0x7fffffff00000000

M shift = 31 right

N mask = 0x00000000fffffffe

N shift = 0 right

Division

Division is comparatively easy. Because SIM_LGP30 is a simulator, we can cheat by computing the floating point quotient of the two operands, converting this to a 32-bit integer, and zeroing the spacer bit. There is an overflow if the quotient has a whole number part.

D scale = 2^{31}

D mask = 0fffffffe

More on LGP-30 Arithmetic

Me: Apart from immediately after an i (input) instruction, is there any time at which the guard bit is significant? Should it be carried through chained operations?

ChatGPT: *This is exactly the question I asked myself when looking at the LGP-30 architecture. My conclusion is that yes, it should be carried through chained operations, but no, it is almost never significant to the programmer.*

Imagine repeatedly adding tiny values. If every instruction discarded the guard bit, you'd effectively be rounding after every operation. If the guard bit is retained, you're effectively carrying one extra bit of precision through the computation until the result is stored. That's exactly what guard digits are for. Many floating-point units still work this way internally.

On Reading Tape Files

The LDP-30 paper tape reader had two modes, 4-bit mode and 6-bit mode. The physical arrangement of channels on a tape was (looking upstream) 612345; thus 4-bit mode used the inner four channels, and 6-bit mode added the outer two channels. Channel 1 was the most significant bit and channel 6 was the least significant bit.

In 4-bit mode, only channels 1, 2, 3 and 4 were read. Each row on the tape represented one hexadecimal digit.

Six bits could encode 64 values – the full gamut of the Friden Flexowriter, so the corresponding convention was that each row represented one character. As a consequence, if a tape was prepared in six-bit mode and read in 4-bit mode, any one of four characters would produce the same input value.

The behaviour of the 'i' instruction changed to match the selected mode. In 4-bit mode, as each row of the tape was read, the value in the accumulator was left-shifted four bits and the new value was placed in the least significant four bits. In six bit mode, the value in the accumulator was left-shifted

six bits and the new value was placed in the least significant six bits. Given that the accumulator was 32 (not 31) bits wide, it could accommodate eight 4-bit values or five 6-bit values.

Although it is easy to make a simple paper tape reader, one must have a tape to read. For almost any computer these days, this is the tricky bit. A simple paper tape reader is easy enough to build, but original paper tapes are hard to come by. It is anything but easy to make even a simple paper tape punch (but see [5] for a really good try). For this reason, most LGP-30 tapes have been converted to text, and saved in text files.

In the process, the distinction between 4-bit and 6-bit mode has been blurred. One might be tempted to declare that any file containing characters other than 0-9, f, g, j, k, q, or w could not have been prepared for use in 4-bit mode, but this overlooks the coding ambiguity which (given the anecdotal evidence) might have been used in some cases to speed up tape preparation.

For use in an LGP-30 simulator, the tape files must be reprocessed – the ASCII or UTF codes within must be translated, either directly or via Friden codes, to binary.

Simulator Implementation

The application “stack” currently comprises:

- Simulator (command level)
- CPU (instruction level)
- Shadow register and memory objects (data level)
- Drum memory and Flexowriter (memory and I/O)

The idea of using shadow objects for program tracing seemed cute enough at the time, but probably added an unnecessary layer of complexity. As it happened, the only objects actively used for tracing are C/pc (because after instruction fetch, C/pc points to the next instruction) and memory (because the operand address of the pending instruction will usually be different from the operand address of the previous instruction). Tracing support could just as easily have been implemented in the CPU.

SIM_LGP30 uses 32-bit variables throughout. A mask is applied by the h and c instructions to reduce the operand to 31 bits plus a zero spacer bit.

The author has attempted to write idiomatic Python, but without any concern for code “density”. Aptly named local variables are used where appropriate for code readability, and it is assumed that the compiler will optimise these out of existence where they are not required.

LGP-30 Programming

For anyone with experience in machine-level programming, the LGP-30 instruction set speaks for itself. It’s easy to come up with a litany of shortcomings which completely overlooks the main design objectives of the LGP-30 – simple and cheap.

Perhaps the best evidence of these shortcomings is the existence of the Floating Point Interpretive System (FPIS) which, according to its own documentation, attempts to create a virtual machine which is more programmer-friendly than the underlying hardware. The documentation warns that using the FPIS incurs a severe performance penalty – proof that you cannot make a silk purse out of a sow’s ear.

Of the CPUs I have programmed at machine level (across a range of eras), the LGP-30 ranks as the second worst in my opinion, the order being:

- Worst: Microchip PIC16F84 (ugly in so many ways)
- LGP-30 (passable at maths, terrible for anything else)
- Intel 8080 (workable, but lacks an indexed addressing mode)
- ATMEL ATmega series (very good for a microcontroller)
- DEC PDP-10 (very powerful)
- Best: DEC PDP-11 (pure elegance in 64 kb)

References

The following references were all “alive” at the time of writing.

1. LGP-30 Programming Manual – https://www.bitsavers.org/pdf/royalPrecision/LGP-30/LGP-30_Programming_Manual_Apr57.pdf
2. LGP-30 Programming Class Notes – http://computermuseum.informatik.uni-stuttgart.de/ftp/lgp30/docs/LGP30_ProgrammingClassNotes.pdf
3. LGP-30 Subroutine Manual – https://www.bitsavers.org/pdf/royalPrecision/LGP-30/LGP-30_Subroutine_Manual_Oct60.pdf
4. University of Stuttgart Computer Museum, LGP-30 artefacts - <http://computermuseum.informatik.uni-stuttgart.de/ftp/lgp30/>
5. Building my own paper tape punch - <https://unimplementedtrap.com/paper-tape-punch/>